Welcome.

Everyone:

- Pull the updates from the course GitHub repo:
  - `cd <46120-PiWE repo>`
  - `git pull origin main`

NB:

- By attending this class, you consent to being recorded. Recording will be shared to this class and possibly other DTU students for training purposes.

46120: Scientific Programming for Wind Energy

Classes, inheritance and OOP (Object Oriented Programming)

Ju Feng

Agenda for today

- Pull new course material ✔

- Classes, inheritance and OOP
- Begin homework

Homework in last week

- **PART 0: Sign up for a final project team (ASAP)**
- **PART 1: Explore one of the pre-defined projects**
  - Discuss with your group, choose one pre-defined project to explore.
  - Write a function to load, parse and plot the provided dataset in inputs, generate at least one figure. Ideally also with test in tests.
  - Think about the structure/architecture of your package.

  Note: the project you choose in this homework doesn't mean you have to stick to that project. Your group can make decision later on which project to work on. If you have a better project idea that your group want to pursue, first check the section on Custom project, and then come talk to us.

  But remember to make a decision early, so that your group has enough time to work on it.

Hand in final project: by Monday, May 4th, at 23:59

A suggested development path

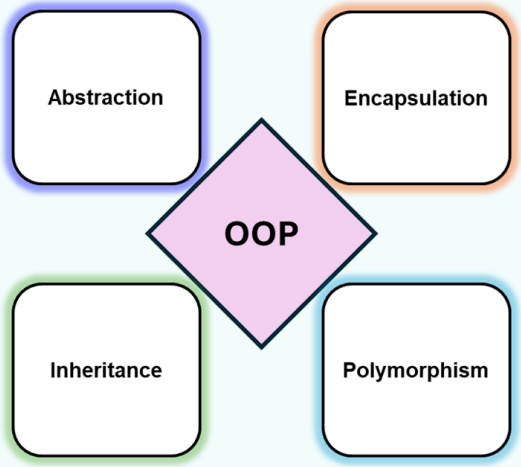
We have a tight schedule, think about your group's development plan. Below is a suggestion to consider:

- **Week 8 and 9**: Understand the project (background, requirements and necessary steps/models/equations), explore the provided data (load, parse, visualize), and consider the preliminary design of the architecture of your package.
- **Week 10**: Implement and refine different parts of the package (including tests).
- **Week 11**: Present your architecture and preliminary results in class, finalize the architecture and polish the code (including tests).
- **Week 12**: Refine the documentation, experiment on different computers (install the package, run the main script, check the test coverage, pylint score), fix all potential issues.

Notes in plenum. (from 2025)

- Think about data structure, you don't have to stick to the provided data format or shape, you can organize it differently based on your system design.
- For example, multiple netCDF4 files are provided in the wind_resource_assessment project, which cover multiple years time series of wind velocity components at two heights for 4 locations. You could consider saving time series with pandas time series format into different files for each locations.
- Interpolation is a common task in scientific computing, you can check `numpy.interp1d` for one-dimensional interpolation, and `scipy.interpolate.interpn` for multidimensional interpolation on regular or rectilinear grids.

Classes, inheritance and OOP



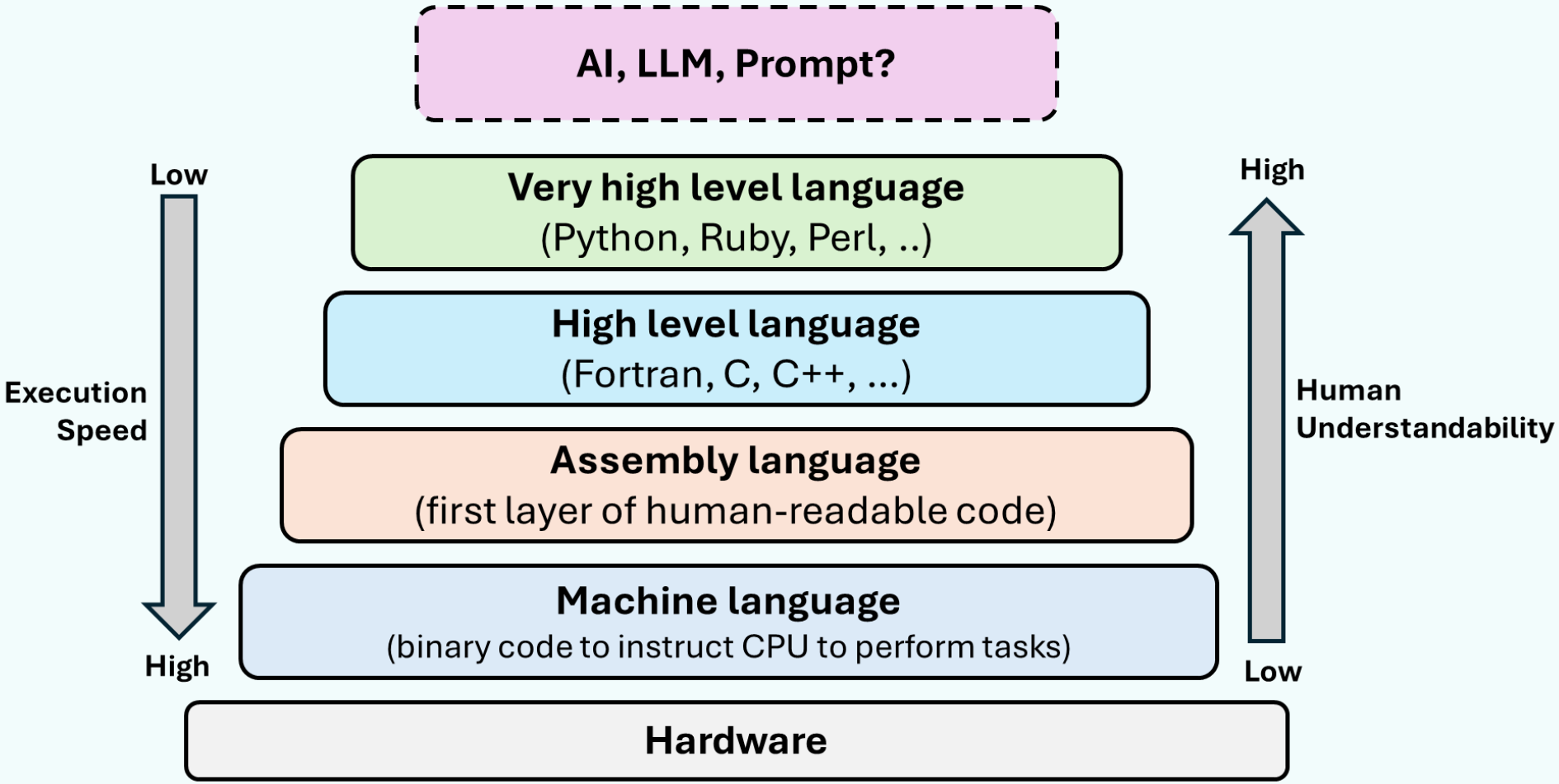
Background

- What is the biggest challenge in software engineering?

> **Managing complexity is the most important technical topic in software development.**
>
> -- Steve McConnell in *Code Complete*

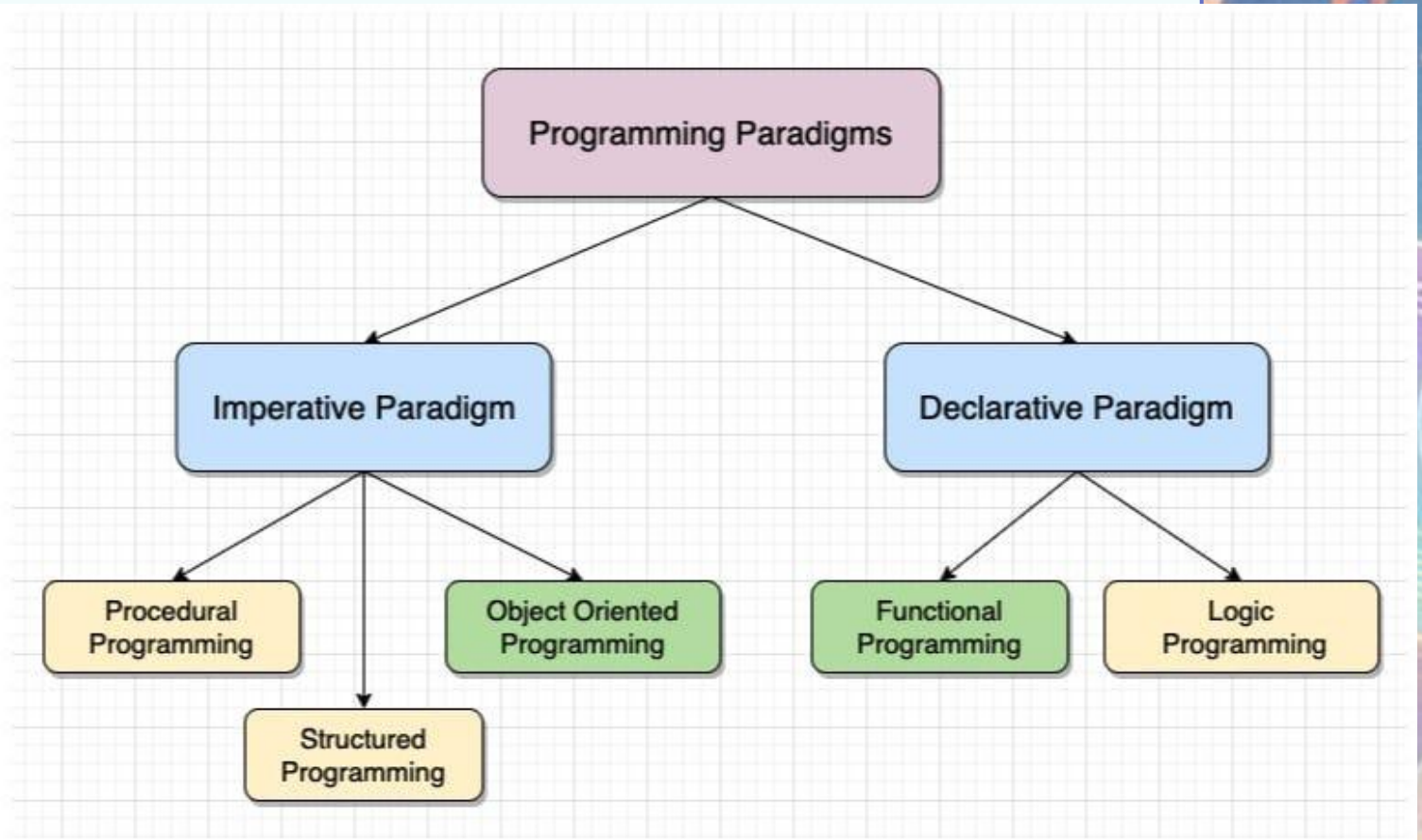
- OOP (object-oriented programming) is an important programming concept/method, also called programming paradigm, that can help a lot in managing complexity.

Levels of programming languages



Programming paradigms

"A programming paradigm is a fundamental approach or style of programming that provides a set of principles, concepts, and techniques for designing and implementing computer programs. It defines the structure, organization, and flow of the code, as well as the methodologies for problem-solving and expressing computations."



Source: Ario Liyan, (2023) What is a programming paradigm?
[link: https://medium.com/@Ariobarxan/what-is-a-programming-paradigm-ec6c5879952b]

Common programming paradigms

Paradigm	Primary Unit	Key Idea	Programming Style
Procedural	Procedure/Function	Sequence of commands that change state	Imperative (How)
Structured	Procedure/Function	Program flow via control structures (no goto)	Imperative (How)
Object-Oriented	Object	Bundling data and behavior into objects	Imperative (How)
Functional	Function	Evaluation of pure functions, no side-effects	Declarative (What)
Logic	Predicate/Rule	Facts and rules for automatic deduction	Declarative (What)

Note: These paradigms are not mutually exclusive. Modern languages like Python are multi-paradigm, allowing you to use a mix of these styles (e.g., writing OOP code with procedural/structured programming elements).

What is OOP?

- OOP stands for Object-Oriented Programming, which is a programming paradigm that organizes software design around the concept of objects. Objects are instances of classes, which encapsulate data and behavior (methods) that operate on that data.

“OOP to me means only messaging, local retention and protection and hiding of state-process, and extreme late-binding of all things.”

-- Alan Kay, who coined the term "object-oriented programming" in 1967

Alan Kay received the Turing Award in 2003 [source]

OOP can be understood primarily as a combination of two programming styles:

- Programming with **user-defined abstract data types** (ADTs) *("local retention and protection and hiding of state-process"*) and
- Programming with **runtime polymorphism** *("messaging ... and extreme late-binding of all things"*).

-- Stefan Ram, *What is object-oriented programming?*

What is OOP?

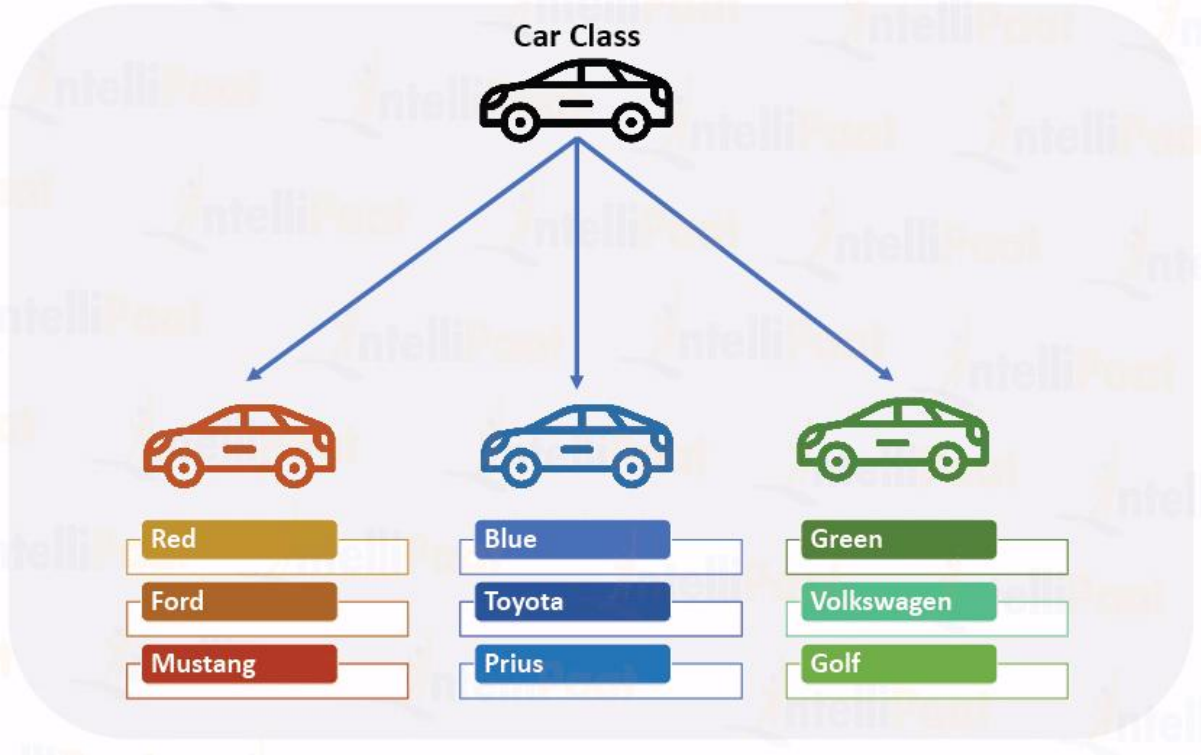


Figure source: https://intellipaat.com/blog/tutorial/python-tutorial/python-classes-and-objects/

- Something is an object (instance) of a type (class) of things.
- A class of things can be commonly characterized by a number of attributes (properties), and can do a number of things (methods).
- Different objects can have different values for the same attributes and do the same thing differently.
- A Class is a blueprint of a group of objects.
- A Class **encapsulate data and behavior** (methods) that operate on that data.

Classes vs. Objects

A key concept in **object-oriented design** is the differentiation between objects and classes. An object is any specific entity that exists in your program at run time. A class is the static thing you look at in the program listing. An object is the dynamic thing with specific values and attributes you see when you run the program. For example, you could declare a class Person that had attributes of name, age, gender, and so on. At run time you would have the objects *nancy*, *hank*, *diane*, *tony*, and so on—that is, specific instances of the class. If you're familiar with database terms, it's the same as the distinction between "schema" and "instance." You could think of the class as the cookie cutter and the object as the cookie.

-- Steve McConnell in *Code Complete*

Usage of Classes

A class is a collection of data and routines that share a cohesive, well-defined responsibility. A class might also be a collection of routines that provides a cohesive set of services even if no common data is involved. A key to being an effective programmer is maximizing the portion of a program that you can safely ignore while working on any one section of code. Classes are the primary tool for accomplishing that objective.

-- Steve McConnell in *Code Complete*

Classes provide a means of **bundling data and functionality together**. Creating a new class creates a new type of object, allowing new instances of that type to be made. Each class instance can have attributes attached to it for maintaining its state. Class instances can also have methods (defined by its class) for modifying its state.

-- Python documentation: https://docs.python.org/3/tutorial/classes.html

Classes in Python

The simplest form of class definition looks like this:

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

```python
class Robot(object):
    def __init__(self, name):
        self.name = name

    def print_name(self):
        print("My name is " + self.name)
```

```java
public class Robot {
    private String name;

    public Robot(String name) {
        this.name = name;
    }

    public void printName() {
        System.out.println("My name is " + this.name);
    }
}
```

A robot class that has a name and can print out its name.

```python
class Robot(object):
    def __init__(self, name):
        self.name = name

    def print_name(self):
        print("My name is " + self.name)
```

```python
class ClassName(BaseClass):
    def __init__(self, …):
        self.attribute1 = …
        …

    def method_1(self, …):
        …
```

- `__init__` is the initialization function (constructor) to initialize/define an object of the class, can be called as: `Object = ClassName(…)` or `ClassName.__init__(Object, …)`
- Inside the class, `self` refers to the class (the object) itself.
- dot operator "`.`" is used to access the attributes or methods of a class as:

  `ClassName.attribute`, `ClassName.method(…)`
- `(object)` is only needed for Python 2 compatibility, can be neglected for Python 3.

Let's try it!

Inheritance

- Inheritance allows us to define a class that inherits all the attributes and methods from another class.
- Parent class is the class being inherited from, also called base class.
- Child class is the class that inherits from another class, also called derived class.
- Child class can extend/override attributes and methods of the Parent class.

```
class ParentClass(object):
    def __init__(self, attribute1):
        self.attribute1 = attribute1

    def method_1(self, para1):
        …
```

```
class ChildClass(ParentClass):
    def __init__(self, attribute1,
                 attribute2):
        ParentClass.__init__(self, attribute1)
        # or use the following line:
        # super().__init__(attribute1)
        self.attribute2 = attribute2

    def method_2(self, para2):
        …
```

Extend Robot to NewRobot

```
class Robot(object):
    def __init__(self, name):
        self.name = name

    def print_name(self):
        print("My name is " + self.name)
```

NewRobot class

- NewRobot class should inherits from the Robot class.
- NewRobot class should have a new attribute: year_of_birth.
- NewRobot class should have a new method: print_age with input current_year .

Let's try it!

Method overriding and polymorphism

- **Method overriding**: a child class provides its own specific implementation of a method defined in the parent.
- **Polymorphsim**: a single method call can behave differently depending on the object type.
- **Benefit**: You can iterate through a list containing various objects — a `BladeSection`, a `TowerSection`, and a `Nacelle`, which may inherit the same base class `TurbineComponent` — and call the same method, `calculate_mass_moment_of_inertia()`, on all of them, but each object executes its own specific calculation.

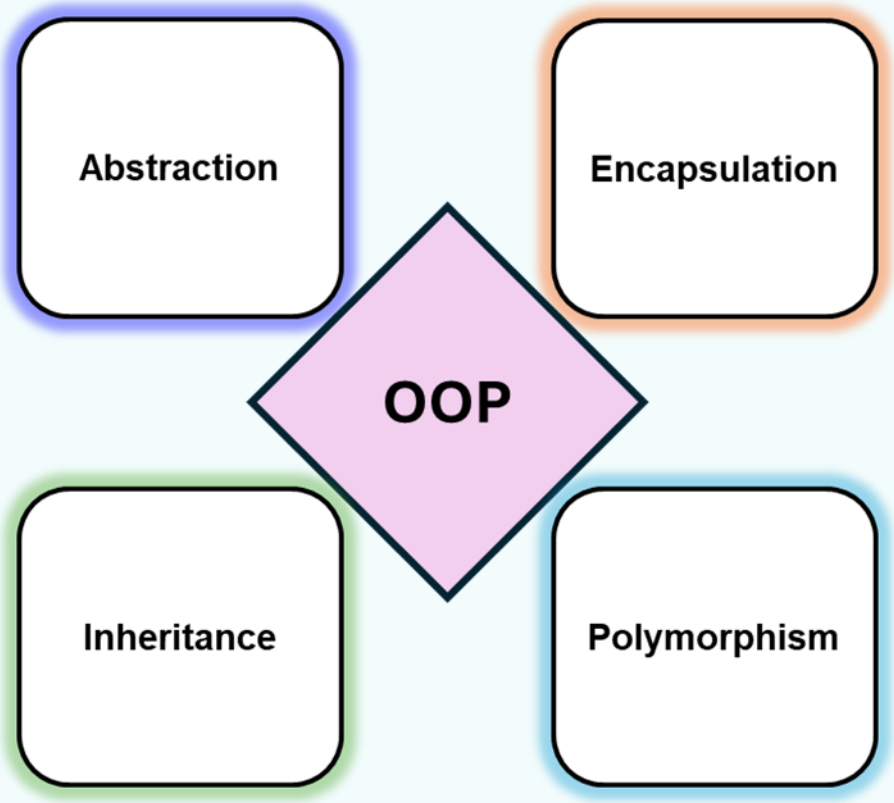
```
class Turbine:
    …
    def get_maintenance_schedule(self):
        return "Standard maintenance every 6 months."

class OffshoreTurbine(Turbine):
    def get_maintenance_schedule(self):
        # Override parent method
        return "Marine-grade maintenance every 12 months."
```

```
# Define two objects with different classes
wt1 = Turbine("T01", 5000)
wt2 = OffshoreTurbine("OS01", 10000, "Monopile")

for turbine in [wt1, wt2]:
    # Call the same method, but get different
    # behave (implementation) for different objects
    print(turbine.get_maintenance_schedule())
```

The Four Pillars of OOP



- **Abstraction**: Hiding complex implementation details and exposing only essential features.
- **Encapsulation**: Bundling data and methods into a single unit (class) while controlling access.
- **Inheritance**: Creating new classes based on existing classes to promote code reuse.
- **Polymorphism**: Objects of different types (classes) can be treated as objects of a common type (class).

Inheritance vs Containment

🧬 Inheritance (IS-A)

*"A Penguin **is a** Bird"*

- **What**: Child class extends parent class
- **When to use**:
  - ➢ True hierarchical relationship
  - ➢ Need polymorphism
  - ➢ Shared core functionality

- ✅ **Pros**: Code reuse, method overriding, Liskov substitution
- ❌ **Cons**: Tight coupling, fragile base class problem
- **Syntax**: `class Child extends Parent` (Java), `class Child(Parent)` (Python)

📦 Containment (HAS-A)

*"A Car **has an** Engine"*

- **What**: Class contains another class as member
- **When to use**:
  - ➢ Modular components
  - ➢ Dynamic behavior changes
  - ➢ Avoid inheritance hierarchies

- ✅ **Pros**: Loose coupling, flexibility, easier testing
- ❌ **Cons**: More boilerplate, indirect access
- **Syntax**: Member field + delegation

When to Choose?

- **Inheritance**: Strict "is-a" relationships with shared core identity
- **Containment**: "Uses-a" or "has-a" relationships, changing requirements

Source: https://chat.deepseek.com/

OOP in Python

- In Python everything is a class in Python. (try use `type()` to check)
- Guido van Rossum has designed the language according to the principle "**first-class everything**". He wrote:

  "*One of my goals for Python was to make it so that all objects were "first class." By this, I meant that I wanted all objects that could be named in the language (e.g., integers, strings, functions, classes, modules, methods, and so on) to have equal status. That is, they can be assigned to variables, placed in lists, stored in dictionaries, passed as arguments, and so forth.*"

  (Blog, The History of Python, February 27, 2009)

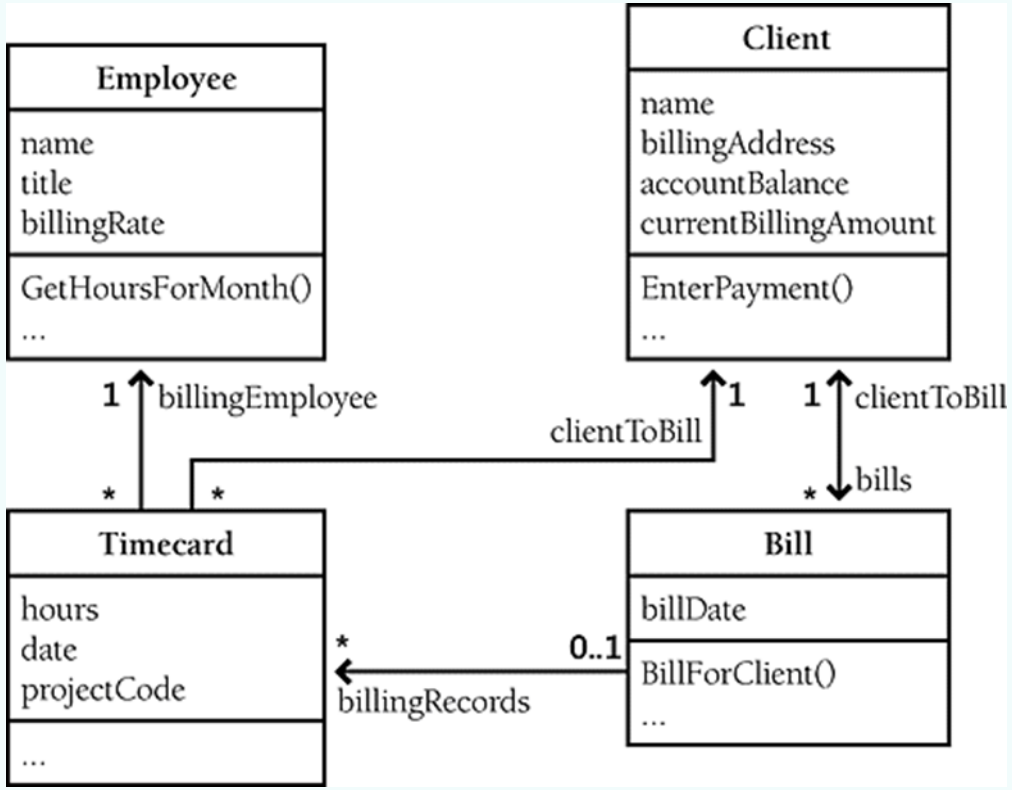
- Everything is a class and treated the same way: functions and methods are values just like lists, integers or floats. Each of these are instances of their corresponding classes.

Source: https://python-course.eu/oop/object-oriented-programming.php

Designing with objects/classes

The steps in designing with objects are:

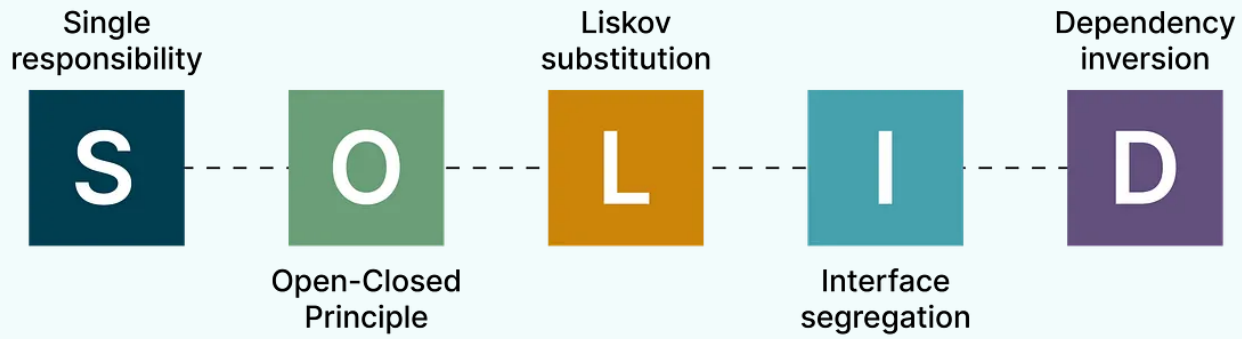
- Identify the objects and their attributes (methods and data).
- Determine what can be done to each object.
- Determine what each object is allowed to do to other objects.
- Determine the parts of each object that will be visible to other objects—which parts will be public and which will be private.
- Define each object's public interface.



A billing system composed of four major objects.

Source: McConnell, Steve. Code complete. Pearson Education, 2004.

The SOLID principles



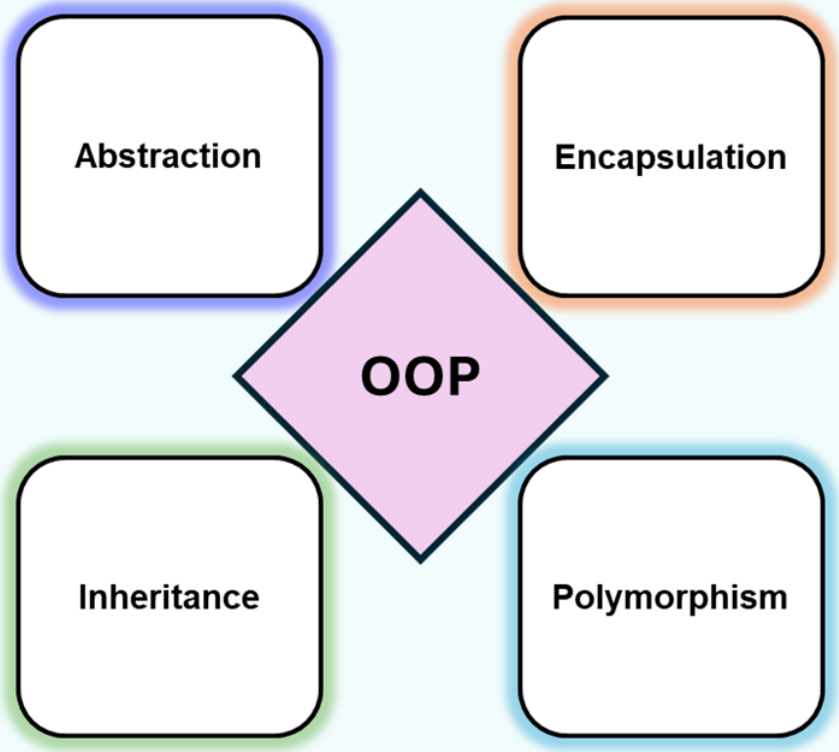
SOLID isn't a rulebook – it's a mindset for developing adaptable software, also not the only or "best" one.

- Five design principles that make object-oriented designs more understandable, flexible, and maintainable: *Originally proposed in: Martin, Robert C. "Design principles and design patterns." Object Mentor 1.34 (2000): 59*
- **Single Responsibility Principle**: A class should have only one reason to change, thus each class should have a single, well-defined responsibility.
- **Liskov Substitution Principle:** Objects of a superclass should be replaceable with objects of a subclass without breaking the application, thus Child classes must maintain the behavior expected by the parent class.

Figure source: https://blog.devops.dev/solid-principles-with-examples-af98d29f20e6

Common OOP Pitfalls

- **Over-engineering**: Creating a deep class hierarchy for a simple analysis script.
- **God Objects**: One class that does everything.
- **Incorrect inheritance**: Using inheritance for "has-a" relationships.
- **Best Advice**: Start simple. Use OOP when you find yourself managing groups of related data and functions.



Procedual programming vs. OOP

Feature	Procedural	OOP
Organization	Functions/Procedures	Objects/Classes
Focus	Logic and Steps	Data and Behavior
Data Security	Data often global (vulnerable)	Data is encapsulated (protected)
Maintenance	High risk of side effects	Changes are isolated (easier)
Modeling	"Doing things"	"Entities that do things"

Use OOP When:

- Modeling real-wold entities with clear state and behavior.
- Building complex systems with multiple interacting components.
- Need to enforce data integrity and relationships.

Stay Procedural When:

- Simple data processing pipelines.
- Mathematical transformations on arrays (NumPy, Pandas).
- One-off scripts and quick analyses.

A balanced view on procedural and object oriented programming

- This course presents both approaches not to advocate for one over the other, but to equip you with multiple problem-solving strategies.
- The mark of an experienced computational scientist/engineer is the ability to select the right tool for the job, whether that tool is procedural, object-oriented, or a thoughtful combination of both.

The goal is to use scientific programming to solve problems, not to pursue paradigm purity.

Class Methods and Static Methods

```python
class WindTurbine(object):
    def __init__(self, id, rated_power):
        self.id = id
        self.rated_power = rated_power

    @classmethod
    def from_config(cls, config_dict):
        """Create turbine from configuration dictionary."""
        return cls(config_dict['id'], config_dict['power'])

    @staticmethod
    def air_density(temperature, pressure):
        """Calculate air density from temp and pressure."""
        return pressure / (287.05 * (temperature + 273.15))
```

```python
# Class method - called on the class
turbine = WindTurbine.from_config(
    {'id': 'WT01', 'power': 15})

# Static method - called on the class
# Like a function in a module
density = WindTurbine.air_density(15, 101.3)
```

Class Methods (`@classmethod`)

- Bound to the class, not the instance
- First parameter: `cls` (the class itself)
- Used for: Alternative constructors, class-level operations

Static Methods (`@staticmethod`)

- Not bound to class or instance
- No special first parameter (no `self` or `cls`)
- Used for: Utility functions related to the class

When to use which?

- `@classmethod`: When you need access to or want to modify the class
- `@staticmethod`: When the function is related to the class but doesn't need class/instance data
- Regular method: When you need access to instance data (`self`)

Special Methods (Dunders)

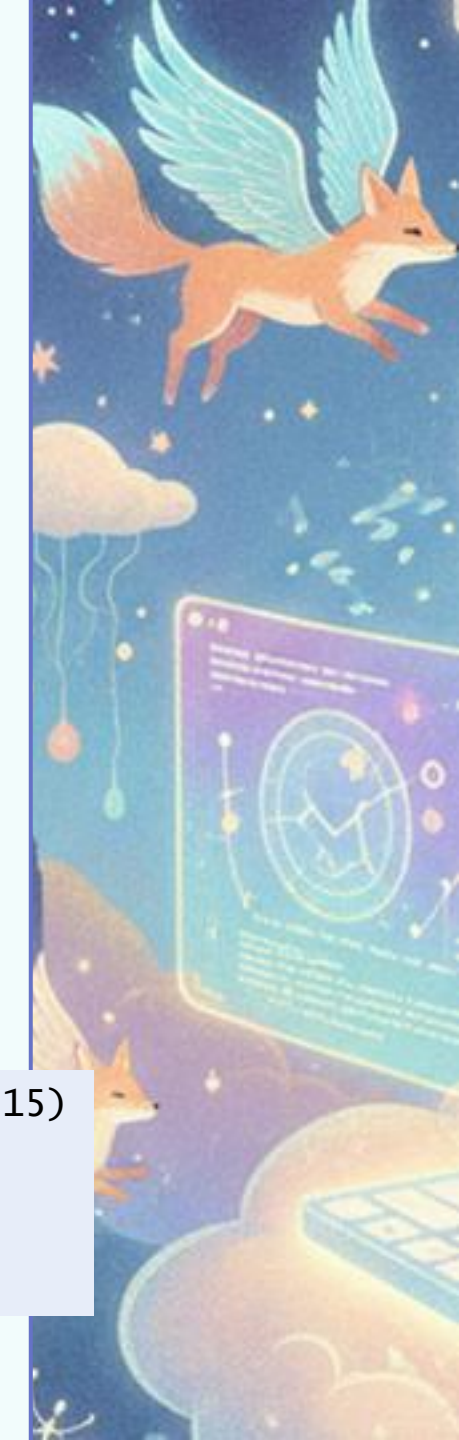
- Dunder: Methods surrounded by double underscores (`__method__`). They allow objects to interact with built-in functions/operators. You need to write them for your classes.
- `__str__`: Returns a user-friendly string representation (what happens when you `print(object)`).
- `__repr__`: Returns an unambiguous, machine-readable string representation (used for debugging/logging).
- You can implement many other special methods (like: `__add__`, `__len__`, `__eq__`) for your class, if needed.

```
class WindTurbine:
    # ... __init__ from before ...

    def __str__(self):
        return f"Turbine {self.id}: {self.rated_power} MW"

    def __repr__(self):
        return f"WindTurbine('{self.id}', {self.rated_power})"
```

```
>>> wt = WindTurbine('WT01', 15)
>>> wt
WindTurbine('WT01', 15)
>>> print(wt)
Turbine WT01: 15 MW
```



Questions?

Homework for this week.

Homework.

- **Homework 0**: Write your own Turbine classes, do it individually.
- **Homework 1**: Write a class for your final project, do it with your group.
- **Homework optional**: Transforming your CodeCamp project with OOP.

Any questions?